



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escola Tècnica Superior d'Enginyeria Informàtica
Universitat Politècnica de València

Manual de Configuración e Instalación

Trabajo Fin de Grado

Grado en Informática Industrial y Robótica

Autor: Lourdes Francés Llimerá

Tutor: Juan Francisco Blanes Noguera

2025-2026

Tabla de contenidos

Tabla de contenidos.....	2
Índice de código.....	3
1. Introducción.....	4
2. Entorno de simulación (Ubuntu 22.04 + ROS 2 Humble)	4
2.1. Máquina virtual (VMware + Ubuntu 22.04).....	4
2.2. ROS 2 Humble	4
2.3. TurtleBot3 y workspace	5
2.4. Framework RAI (fork modificado)	5
2.5. Entorno virtual de Python (laric_env).....	6
2.6. Clave de API e idioma del LLM.....	6
2.7. Compilación del paquete y primer arranque	6
3. Entorno del robot real (Ubuntu 24.04 + ROS 2 Jazzy).....	7
3.1. Equipo con Ubuntu 24.04	7
3.2. ROS 2 Jazzy.....	8
3.3. Workspace de LARIC	8
3.4. Framework RAI (fork modificado)	9
3.5. Entorno virtual de Python (laric_env).....	9
3.6. Clave de API, idioma del LLM y conmutadores	10
3.7. Compilación del paquete.....	10
3.8. Red: comunicación entre la VM y el ROSbot 3.....	10
3.9. Stack de navegación a bordo del robot	11
3.10. Creación del mapa y extracción de ubicaciones.....	11
3.11. Arranque y parada del cliente.....	11

Índice de código

Código 1. Modificación del archivo de configuración .vmx (Humble).	4
Código 2. Instalación ROS 2 Humble.....	5
Código 3. Verificación ROS 2 Humble.	5
Código 4. Clonación del workspace (Humble).	5
Código 5. Comando alternativo para clonar el repositorio (Humble).	5
Código 6. Prueba del simulador (Humble).	5
Código 7. Clonación del framework RAI (Humble).	6
Código 8. Verificación RAI (Humble).	6
Código 9. Creación entorno virtual (Humble).....	6
Código 10. Export de clave API (Humble).	6
Código 11. Compilación del paquete y primer arranque (Humble).....	6
Código 12. Modificación del archivo de configuración .vmx (Jazzy).	7
Código 13. Desactivación de la suspensión automática en WirePlumber (Jazzy).	8
Código 14. Instalación ROS 2 Jazzy.	8
Código 15. Verificación ROS 2 Jazzy.	8
Código 16. Clonación del workspace (Jazzy).	9
Código 17. Comando alternativo para clonar el repositorio (Jazzy).	9
Código 18. Clonación del framework RAI (Jazzy).	9
Código 19. Verificación RAI (Jazzy).	9
Código 20. Creación entorno virtual (Jazzy).....	10
Código 21. Export de clave API y ROS DOMAIN ID (Jazzy).....	10
Código 22. Compilación del paquete (Jazzy).	10
Código 23. Configuración de red (Jazzy).....	11
Código 24. Arranque y parada del cliente (Jazzy).	12



1. Introducción

Este manual describe la preparación completa del entorno de LARIC. El sistema se despliega en dos entornos claramente diferenciados que conviene no mezclar:

- **Simulación:** Ubuntu 22.04 LTS con ROS 2 Humble y Gazebo. Se eligió Humble por los problemas de rendimiento de Gazebo con Ubuntu 24.04 y con la aceleración gráfica de la máquina virtual.
- **Robot real:** Ubuntu 24.04 LTS con ROS 2 Jazzy. Se eligió porque el Husarion ROSbot 3 funciona sobre Jazzy y el cliente debe coincidir con la distribución del robot.

Ambos entornos comparten el mismo código (laric_ws) y se diferencian esencialmente por los conmutadores de config.py (is_real_robot, is_from_lab) y por los scripts de arranque (start_simulation.sh frente a start_real.sh).

2. Entorno de simulación (Ubuntu 22.04 + ROS 2 Humble)

2.1. Máquina virtual (VMware + Ubuntu 22.04)

1. Instalar [VMware Workstation Pro](https://www.vmware.com/es/try-vmware-workstation-pro.html) (gratuito para uso personal desde support.broadcom.com).
2. Descargar la imagen ubuntu-22-04.x-desktop-amd64.iso.
3. Crear la VM: Mínimo 50 GB de disco (split), 4096 MB de RAM (8192 recomendado), 2 cores por procesador (4 recomendado).
4. Activar “Accelerate 3D graphics” con la máxima memoria gráfica posible; sin ello RViz no arranca correctamente.

Para evitar avisos de ALSA del driver de audio antiguo de VMware, con la VM apagada, añadir al archivo de configuración .vmx la línea:

```
sound.virtualDev = "hdaudio"
```

Código 1. Modificación del archivo de configuración .vmx (Humble).

2.2. ROS 2 Humble

```
sudo apt update && sudo apt upgrade -y

sudo apt install git curl gnupg2 lsb-release python3-pip python3.10-venv terminator -y

# Locale
sudo locale-gen en_US en_US.UTF-8

sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

# Repositorio de ROS 2
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
-o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
```

```
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
sudo apt update
sudo apt install ros-humble-desktop ros-dev-tools -y
echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc && source ~/.bashrc
```

Código 2. Instalación ROS 2 Humble.

Verificación: en dos terminales distintas, ejecutar respectivamente

```
ros2 run demo_nodes_cpp talker # Terminal 1
ros2 run demo_nodes_py listener # Terminal 2
```

Código 3. Verificación ROS 2 Humble.

Si el listener recibe los mensajes, ROS 2 está instalado correctamente.

2.3. TurtleBot3 y workspace

```
sudo apt install ros-humble-turtlebot3* -y
echo 'export TURTLEBOT3_MODEL=waffle_pi' >> ~/.bashrc && source ~/.bashrc
cd ~
git clone https://gitlab.ai2.upv.es/rai\_ai2/laric
cp -r ~/LARIC-TFG/laric_ws ~/laric_ws
cd ~/laric_ws && colcon build
echo 'source ~/laric_ws/install/setup.bash' >> ~/.bashrc && source ~/.bashrc
```

Código 4. Clonación del workspace (Humble).

Se copia laric_ws al directorio del usuario para que las rutas absolutas de los scripts funcionen y para aislar build/ e install/ del control de versiones.

Alternativamente, se puede clonar el repositorio mediante el comando siguiente:

```
git clone https://github.com/lourdesfll29-maker/LARIC-TFG.git
```

Código 5. Comando alternativo para clonar el repositorio (Humble).

Prueba del simulador:

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Código 6. Prueba del simulador (Humble).

2.4. Framework RAI (fork modificado)

LARIC usa una versión modificada de RAI. La modificación corrige el tratamiento del resultado de las acciones de Nav2 (véase el Manual de Programación). Es imprescindible instalar el fork de la autora, no el repositorio original.

```
pip3 install uv
cd ~ && git clone https://github.com/lourdesfll29-maker/rai.git
cd rai
sudo ~/.local/bin/uv pip install --system -e src/rai_core
```

```
sudo ~/.local/bin/uv pip install --system -e src/rai_s2s
```

Código 7. Clonación del framework RAI (Humble).

Verificación:

```
python3 -c "from rai.communication.ros2 import ROS2Connector; print('RAI OK')"
```

Código 8. Verificación RAI (Humble).

Es normal que aparezcan avisos de pydub (ffmpeg) y de rai_interfaces; se ignoran. Si al final imprime "RAI OK", la instalación es correcta.

2.5. Entorno virtual de Python (laric_env)

```
cd ~/laric_ws
```

```
python3 -m venv --system-site-packages laric_env
```

```
source laric_env/bin/activate
```

```
pip install -r requirements.txt
```

```
sudo apt install portaudio19-dev python3-pyaudio ffmpeg -y
```

Código 9. Creación entorno virtual (Humble).

Se usa `--system-site-packages` para que el entorno aísle las librerías de IA (ecosistema LangChain) pero conserve el acceso a la instalación base de ROS 2. Los avisos de conflictos de dependencias de pip durante la instalación son esperables y pueden ignorarse si la última línea indica "Successfully installed...".

2.6. Clave de API e idioma del LLM

La credencial del LLM se gestiona mediante variable de entorno:

```
echo 'export GROQ_API_KEY="gsk_..."' >> ~/.bashrc
```

```
source ~/.bashrc
```

Código 10. Export de clave API (Humble).

Alternativamente, poner `"is_from_lab = True"` en `config.py` para usar el servidor Ollama del ai2 de la UPV (requiere acceso a la red de la universidad).

2.7. Compilación del paquete y primer arranque

```
cd ~/laric_ws && source laric_env/bin/activate
```

```
colcon build --symlink-install --packages-select laric_core
```

```
source install/setup.bash
```

```
chmod +x ~/laric_ws/scripts/*.sh
```

```
./scripts/start_simulation.sh
```

Código 11. Compilación del paquete y primer arranque (Humble).

El arranque lanza cuatro procesos en orden: Gazebo (con una espera de 10 s), Nav2 con RViz (5 s), la HMI (3 s) y el agente. Cada proceso registra su salida en `laric_ws/laric_logs/<marca>/`, y la HMI muestra el estado y los errores de arranque en vivo. Antes de enviar comandos, realizar el "2D Pose Estimate" en RViz. Con todo

funcionando, conviene hacer un snapshot de la VM (por ejemplo, "LARIC-OK") para poder restaurar el entorno.

3. Entorno del robot real (Ubuntu 24.04 + ROS 2 Jazzy)

El robot real es un Husarion ROSbot 3 que ejecuta a bordo Ubuntu y ROS 2 Jazzy. El portátil (o la VM) actúa solo como cliente ROS 2: ejecuta la HMI y el agente, mientras que Nav2, el snap del RPLIDAR y el resto de la percepción corren en el propio robot (la localización se hace contra un mapa estático, con SLAM desactivado).

Esta parte recoge la instalación completa del cliente para Ubuntu 24.04 con ROS 2 Jazzy. Aunque los pasos son análogos a los de la simulación, se reproducen aquí enteros.

3.1. Equipo con Ubuntu 24.04

El cliente puede ser un equipo con Ubuntu 24.04 nativo o una máquina virtual. Si es una VM, necesita dos tarjetas de red (véase el apartado 3.8). A diferencia de la simulación, el cliente no ejecuta Gazebo ni RViz (la visualización la sirve el robot por Foxglove), por lo que la aceleración 3D no es imprescindible.

0. Si se usa VM: Instalar [VMware Workstation Pro](https://www.vmware.com/es/workstation-pro) (gratuito para uso personal desde support.broadcom.com).
1. Descargar la imagen ubuntu-24-04.x-desktop-amd64.iso desde ubuntu.com.
2. Crear la VM: Mínimo 50 GB de disco (split), 4096 MB de RAM (8192 recomendado), 2 cores por procesador (4 recomendado).

Si se usa una máquina virtual (VM) con distribuciones modernas (como Ubuntu 24.04) y el audio se escucha entrecortado bajo carga de trabajo, se recomienda forzar el uso de un controlador de audio más ligero y compatible. Con la VM apagada, añadir al archivo de configuración .vmx la línea:

```
sound.virtualDev = "es1371"
```

Código 12. Modificación del archivo de configuración .vmx (Jazzy).

Adicionalmente, para evitar que el audio se entrecorte al inicio de cada reproducción debido al ahorro de energía del sistema, se debe ejecutar el siguiente comando en la terminal de la VM:

```
mkdir -p ~/.config/wireplumber/main.lua.d/
cat << 'EOF' > ~/.config/wireplumber/main.lua.d/50-disable-suspend.lua
table.insert (alsa_monitor.rules, {
  matches = {
    {
      { "node.name", "matches", "alsa_output.*" },
    },
  },
  apply_properties = {
    ["session.suspend-timeout-seconds"] = 0,
  },
})
EOF
systemctl --user restart wireplumber
```

Código 13. Desactivación de la suspensión automática en WirePlumber (Jazzy).

El cambio al controlador es1371 emula una tarjeta de sonido clásica que es mucho más ligera y tolerante con los picos de uso de la CPU en entornos virtualizados, eliminando las interferencias de PipeWire. Por su parte, el script de WirePlumber desactiva la suspensión automática (`suspend-timeout-seconds = 0`), obligando a la tarjeta de sonido virtual a permanecer siempre activa. Esto evita el retraso y el pequeño tartamudeo que ocurre durante los primeros segundos cuando el sistema intenta "despertar" el driver de audio.

3.2. ROS 2 Jazzy

```
sudo apt update && sudo apt upgrade -y

sudo apt install open-vm-tools open-vm-tools-desktop -y

sudo apt install git curl gnupg2 lsb-release python3-pip python3.12-venv terminator -y

# Locale
sudo locale-gen en_US en_US.UTF-8

sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

# Repositorio de ROS 2
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
-o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME)
main" \
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

sudo apt update

sudo apt install ros-jazzy-desktop ros-dev-tools -y

echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc && source ~/.bashrc
```

Código 14. Instalación ROS 2 Jazzy.

Verificación: en dos terminales distintas, ejecutar respectivamente

```
ros2 run demo_nodes_cpp talker # Terminal 1

ros2 run demo_nodes_py listener # Terminal 2
```

Código 15. Verificación ROS 2 Jazzy.

Si el listener recibe los mensajes, ROS 2 está instalado correctamente.

3.3. Workspace de LARIC

La instalación de TurtleBot3 es opcional en el cliente real (no se usa Gazebo); el workspace, en cambio, es necesario:

```
# TurtleBot3 (opcional en el cliente real):
sudo apt install ros-jazzy-turtlebot3* -y

echo 'export TURTLEBOT3_MODEL=waffle_pi' >> ~/.bashrc && source ~/.bashrc

cd ~

git clone https://gitlab.ai2.upv.es/rai_ai2/laric

cp -r ~/LARIC-TFG/laric_ws ~/laric_ws

cd ~/laric_ws && colcon build

echo 'source ~/laric_ws/install/setup.bash' >> ~/.bashrc && source ~/.bashrc
```

Código 16. Clonación del workspace (Jazzy).

Se copia laric_ws al directorio del usuario para que las rutas absolutas de los scripts funcionen y para aislar build/ e install/ del control de versiones.

Alternativamente, se puede clonar el repositorio mediante el comando siguiente:

```
git clone https://github.com/lourdesfll29-maker/LARIC-TFG.git
```

Código 17. Comando alternativo para clonar el repositorio (Jazzy).

3.4. Framework RAI (fork modificado)

Igual que en la simulación, LARIC usa una versión modificada de RAI que corrige el tratamiento del resultado de las acciones de Nav2 (véase el Manual de Programación). Es imprescindible instalar el fork de la autora, no el repositorio original.

```
curl -Lsf https://astral.sh/uv/install.sh | sh

cd ~ && git clone https://github.com/lourdesfll29-maker/rai.git

cd rai

sudo ~/.local/bin/uv pip install --system --break-system-packages -e src/rai_core

sudo ~/.local/bin/uv pip install --system --break-system-packages -e src/rai_s2s
```

Código 18. Clonación del framework RAI (Jazzy).

Verificación:

```
python3 -c "from rai.communication.ros2 import ROS2Connector; print('RAI OK')"
```

Código 19. Verificación RAI (Jazzy).

Es normal que aparezcan avisos de pydub (ffmpeg) y de rai_interfaces; se ignoran. Si al final imprime "RAI OK", la instalación es correcta.

3.5. Entorno virtual de Python (laric_env)

```
cd ~/laric_ws

python3 -m venv --system-site-packages laric_env

source laric_env/bin/activate
```

```
pip install -r requirements.txt

sudo apt install portaudio19-dev python3-pyaudio ffmpeg -y
```

Código 20. Creación entorno virtual (Jazzy).

Se usa `--system-site-packages` para que el entorno aísle las librerías de IA (ecosistema LangChain) pero conserve el acceso a la instalación base de ROS 2. Los avisos de conflictos de dependencias de pip durante la instalación son esperables y pueden ignorarse si la última línea indica "Successfully installed...".

3.6. Clave de API, idioma del LLM y conmutadores

La credencial del LLM se gestiona mediante variable de entorno:

```
echo 'export GROQ_API_KEY="gsk_..." ' >> ~/.bashrc

echo 'export ROS_DOMAIN_ID=0' >> ~/.bashrc

source ~/.bashrc
```

Código 21. Export de clave API y ROS DOMAIN ID (Jazzy).

En `config.py` hay que poner `is_real_robot = True` (selecciona el mapa semántico real y el script de apagado `stop_real.sh`). El conmutador `is_from_lab` selecciona el LLM: `True` usa el servidor Ollama del ai2 de la UPV (requiere acceso a la red de la universidad); `False` usa Groq y necesita la `GROQ_API_KEY` anterior.

3.7. Compilación del paquete

```
cd ~/laric_ws && source laric_env/bin/activate

colcon build --symlink-install --packages-select laric_core

source install/setup.bash

chmod +x ~/laric_ws/scripts/*.sh
```

Código 22. Compilación del paquete (Jazzy).

El arranque del cliente se describe en el apartado 3.11, una vez configurada la red y con el robot ya navegando.

3.8. Red: comunicación entre la VM y el ROSbot 3

La red de la UPV exige credenciales, lo que impide que el robot se conecte directamente a ella. La solución adoptada usa un router local (`MERCUSYS_ai2`) sin salida a Internet para la comunicación ROS 2, y aprovecha que la VM puede tener dos tarjetas de red:

- **Tarjeta 1 (WiFi a UPVNET):** Da al PC acceso a la API externa (Groq) y al servidor Ollama del laboratorio.
- **Tarjeta 2 (Ethernet al router MERCUSYS_ai2):** Red local a la que el ROSbot se conecta por WiFi. Por aquí circula todo el tráfico ROS 2 (DDS) entre la VM y el robot.

De este modo, la VM puede hablar con el robot por la red local y, a la vez, acceder al LLM por la red de la universidad. La configuración de red para ROS 2 (ya incluida en `start_real.sh`) es:

```
export ROS_DOMAIN_ID=0
export ROS_AUTOMATIC_DISCOVERY_RANGE=SUBNET # descubrimiento DDS en la LAN
export ROS_STATIC_PEERS="192.168.1.46;192.168.1.102" # ROSbot;cliente
```

Código 23. Configuración de red (Jazzy).

El ROSbot tiene la IP 192.168.1.46 en la red MERCUSYS_ai2. Originalmente el robot necesitaba conexión WiFi a Internet en cada arranque de navegación; se modificó el snap de navegación para que no realice una descarga (pull) en cada ejecución, de modo que funcione en la red local sin Internet.

El router del laboratorio no reenvía de forma fiable el tráfico multicast entre clientes WiFi, del que depende el descubrimiento por SUBNET. Cuando falla, los nodos del robot se ven en `ros2 node list` pero los mensajes publicados una sola vez (como `/initialpose` desde Foxglove) no llegan al cliente. La variable `ROS_STATIC_PEERS` lista las IPs de los participantes para que DDS los descubra por unicast, sin depender del multicast.

3.9. Stack de navegación a bordo del robot

En el ordenador de a bordo del ROSbot, dentro de `~/rosbot-autonomy`, se arranca la navegación con "just start-navigation" y se detiene con "just stop". El robot expone, mediante snaps:

- **husarion-webui:** Interfaz web Foxglove en `http://192.168.1.46:8080/ui` para visualizar el mapa y publicar la pose inicial.
- **husarion-rplidar:** Driver del LIDAR.
- **husarion-depthai:** Driver de la cámara de profundidad.
- El stack de navegación de Nav2 (con SLAM desactivado en operación; localización contra el mapa estático).

3.10. Creación del mapa y extracción de ubicaciones

La primera vez se arranca la navegación con `SLAM:=True` para mapear el entorno conduciendo el robot. Una vez guardado el mapa, se cambia a `SLAM:=False` para trabajar siempre contra un mapa estático (necesario para una localización estable). Con el mapa ya creado, se obtienen las coordenadas de las ubicaciones del mapa semántico pinchando puntos en la interfaz web; el robot publica esos puntos en el tópico `/clicked_point`, de donde se copian las coordenadas a `KNOWN_LOCATIONS_REAL` en `config.py`.

La interfaz web del robot puede usarse desde un PC (conectado al router local) sin VM accediendo por IP, ya que publica los tópicos tanto al robot como a la VM. Para más detalle del funcionamiento de la navegación con los snaps `husarion-webui`, `depthai` y `rplidar`, consultar la documentación de Husarion.

3.11. Arranque y parada del cliente

```
# En el robot: 'just start-navigation' dentro de ~/rosbot-autonomy

# En el PC/VM (con el robot navegando):
cd ~/laric_ws && ./scripts/start_real.sh
```

```
# Para apagar solo el cliente (la navegación del robot sigue activa):  
./scripts/stop_real.sh  
  
# En el robot: 'just stop' dentro de ~/rostop-autonomy
```

Código 24. Arranque y parada del cliente (Jazzy).

start_real.sh hace ping al robot, lanza la HMI y el agente, y recuerda fijar la pose inicial desde Foxglove antes de enviar comandos. No abre RViz, porque la visualización ya la sirve el robot vía Foxglove. Con todo funcionando, conviene hacer un snapshot de la VM (por ejemplo, "LARIC-OK") para poder restaurar el entorno.